ScrollView- When and How to Use it

ScrollView is a scrollable container that renders all its children at once. It wraps content that's larger than the screen

```
import { ScrollView } from 'react-native';

<ScrollView>
    <View>---</View>
    <View>---</View>
    <View>---</View>
</ScrollView>
```

When to Use ScrollView

- → Small, fixed amount of content
- → Mixed content type
- → Need nested Scrolling
- → content is not from array/data
- → Less than 20-30 items

When not to use:

- → Large lists (100+ items)
- → Data from arrays
- → Need performance
- → Infinite Scrolling

Common ScrollView Props

```
<ScrollView
    horizontal={false} // Vertical (default) or horizontal
    showsVerticalScrollIndicator={true}
    ShowsHorizontalScrollIndicator={false}
    contentContainerStyle={styles.content}
                        // Style inner content
    bounces={true}  // Bounce at edge (iOS)
    scrollEnabled={true}  // Enable/Disable
                          Scrolling
    pagingEnabled={false}  // Snap to pages
    onScroll={(event) => {}}  // Scroll event handler
>
  {/* Content */}
</ScrollView>
```

FlatList – Performant List Rendering

FlatList is a performant, scrollable list component that renders items lazily (only visible items + few offscreen)

```
import { FlatList } from 'react-native';
<FlatList
    data={arrayOfData}  // Array data required
    renderItem={({item}) => <YourComponent
                                item={item} />}
    KeyExtractor={(item) => item.id}
                        // unique key for each item
/>
```

ScrollView

→ Renders ALL items at once.
→ Memory intensive
→ Slow with Large Lists
→ Simple to use
→ For mixed content

FlatList

→ Render only visible items.
→ Memory efficient
→ Fast with large List
→ Require data array
→ For uniform lists

Performance Benefits:

→ **Lazy rendering** - Only renders what's on screen
→ **Recycling** - Reuses components as you scro
→ **Windowing** - Keeps few items above/belo viewport
→ **Optimized** - Built for mobile performance

Common FlatList Props:

```
<FlatList
  data={data}
  renderItem={({item, index}) => <Item />}
  keyExtractor={(item) => item.id.toString()}
  //Layout
  numColumns={1}
```

```
// Components
ListHeaderComponent={<Header/>}
ListFooterComponent={<Footer/>}
ListEmptyComponent={<Empty/>} // When data is emp
ItemSeperatorComponent={<Sep/>} // B/W Items

// Performance
initialNumToRender={10} // items to render initially
maxToRenderPerBatch={10} // Items per batch
windowSize={10} // ViewPort multiplier
removeClippedSubviews={true} // Memory opt

// Refresh
refreshing={false}
onRefresh={()=>{}}
onEndReach={()=>{}}
onEndReachedThreshold={0.5}
/>
```

renderItem Function

```
const renderItem=({item, index})=>{
  return(
    <View style={style.item}>
    <Text>{item.name}</Text>
    <Text>{item.price}</Text>
```

```
<FlatList
    data={product}
    renderItem={renderItem}
    keyExtractor={(item) => item.id}
/>
```

KeyExtractor Importance:

```
keyExtractor={(item) => item.id.toString()}

keyExtractor={(item) => `${item.id}-${itemname}`}

X keyExtractor={(item, index) => index.toString()}
```

SectionList - Group Data

```
<SectionList
    sections={sectionData}
    renderItem={({item}) => <Text>
                                {item}</Text>

    renderSectionHeader

        keyExtractor
```

When to use:

- data is naturally grouped
- Need section headers